Problem 1: Influencer & KOL Discovery via Centrality Metrics

Objective and dataset

The most influential accounts in the Telugu tweet dataset are YS Jagan (@ysjagan), Chandrababu Naidu (@ncbn) and Pawan Kalyan (@pawankalyan), ranked by in-degree and PageRank on a user mention network of 30,648 users and 72,711 edges.

Objective: build a directed User Mention Network from the tweet text and identify key opinion leaders (KOLs) using Degree Centrality, Betweenness Centrality and PageRank, then visualise the results in Neo4j.

Dataset: `raw_data_te.jsonl`, the Telugu-language tweet file provided for the course, in JSON Lines format (one tweet per line). Each record holds `tid`, `author_id`, `author_screen_name`, `tweet_text`, `lang`, `datetime` and `location`. Only `author_screen_name` and `tweet_text` are used for this problem.

Tech stack: Python 3.12, Pandas, NetworkX, Neo4j Desktop 2.3.1 (Neo4j Browser and Bloom).

Size: 82,533 tweets, of which 60,474 (73.3%) contain at least one @mention.

Preprocessing and graph construction

Each tweet adds one directed edge from its author to every user it mentions, giving a sparse graph of 30,648 nodes and 72,711 weighted edges.

Mention extraction. Mentions are found with the regex `@([A-Za-z0-9_]+)`. Twitter handles only contain letters, digits and underscores, so the match stops cleanly before any Telugu text (`@PawanKalyan` గారు gives `PawanKalyan`). Retweets (`RT @user:`) count as mentions, since retweeting is a form of amplification.

Cleaning rules:

- Handles are lowercased, because Twitter handles are case-insensitive (`@NCBN` and `@ncbn` are one user).
- Self-mentions (author = mentioned user) are dropped.
- Repeated mentions between the same pair are merged into one edge, and the count is stored as the edge `weight`.
- Tweets with no mentions contribute no edges.

Example edges from the data:

Author	Tweet (shortened)	Edges created
teampowersena	… JanaSena Chief Sri @PawanKalyan గారు	teampowersena → pawankalyan
im_yogeshm	RT @AmitShah: …	im_yogeshm → amitshah
iamravipatinani	RT @ncbn: … @AyyannaPatruduC గారికి …	iamravipatinani → ncbn, iamravipatinani → ayyannapatruduc
tandansagar464	(no mentions)	none

Graph statistics:

Property	Value
Type	Directed, weighted (`nx.DiGraph`)
Nodes (users)	30,648
Edges (author → mentioned user)	72,711
Average out-degree	2.37
Density	7.74×10^{-5}

The code is split into two scripts: `build_graph.py` builds the graph and saves it to `mention_graph.pkl`, and `centrality.py` loads it, computes the scores and writes `centrality_scores.csv`. A third script, `export_edges.py`, writes `edges.csv` for Neo4j.

Centrality measures used

Three measures capture three different kinds of influence: popularity (in-degree), bridging (betweenness) and quality-weighted popularity (PageRank).

In-degree centrality is the share of other users who mention a user. In a mention network, being mentioned is the signal of influence, so in-degree is used rather than total degree.

```
None
C_{in}(v) = \frac{\deg^{-}(v)}{n - 1}
```

Betweenness centrality is the fraction of all shortest paths that pass through a user. High betweenness marks a bridge between otherwise separate groups.

```
None
C_B(v) = \sum_{s \neq v \neq t}
\frac{\sigma_{st}(v)}{\sigma_{st}}
```

Exact betweenness costs O(nm), which is slow for 30,648 nodes. It was approximated with `k = 500` sampled source nodes and `seed = 42` for reproducibility.

PageRank scores a user highly when they are mentioned by users who are themselves highly scored. Edge weights are used, so repeated mentions count more. The damping factor is NetworkX's default, d = 0.85.

```
None
PR(v) = \frac{1 - d}{n} + d \sum_{u \to v} \frac{w_{uv}}{\sum_{x}
w_{ux}} \, PR(u)
```

Measure	NetworkX call	Influence it captures
In-degree	`nx.in_degree_centrality(G)`	How many users mention you

Betweenness	`nx.betweenness_centrality(G, k=500, seed=42)`	How often you connect other users
PageRank	`nx.pagerank(G, weight="weight")`	Whether influential or frequent mentioners point to you

Results

In-degree and PageRank agree on the top four users, while betweenness picks out a completely different set of ordinary accounts.

Top 10 by in-degree and PageRank. The "Mentioned by" column converts in-degree centrality back to a count: C_in × (n − 1), with n − 1 = 30,647.

Rank	In-degree user	In-degree centrality	Mentioned by (users)	PageRank user	PageRank
1	ysjagan	0.1059	3,245	ysjagan	0.0351
2	ncbn	0.0929	2,848	ncbn	0.0326
3	pawankalyan	0.0843	2,582	pawankalyan	0.0238
4	narendramodi	0.0689	2,113	narendramodi	0.0229
5	janasenaparty	0.0508	1,556	bjp4telangana	0.0228
6	ktrtrs	0.0492	1,508	janasenaparty	0.0101
7	kchirutweets	0.0285	873	ktrtrs	0.0101

8	andhrapradeshcm	0.0269	823	andhrapradeshcm	0.0100
9	amitshah	0.0265	811	amitshah	0.0087
10	trspartyonline	0.0219	670	kchirutweets	0.0056

Top 10 by betweenness (sampled, k = 500):

Rank	User	Betweenness
1	saibabasanthos1	0.001043
2	praja_snklpm	0.000996
3	smahaboob17	0.000674
4	raoivb	0.000502
5	shivare46619067	0.000502
6	syednoorjahan3	0.000501
7	surya67961017	0.000484
8	mpraveenreddy13	0.000470

9	710b50086428483f	0.000453
10	iamrameshsriram	0.000437

Full scores for all 30,648 users are in `centrality_scores.csv`.

Neo4j visualization

All 30,648 users and 72,711 MENTIONS relationships were loaded into Neo4j, and Bloom shows the top 10 PageRank users as hubs with their 30 strongest mentioners each.

Import. `centrality_scores.csv` (users + scores) and `edges.csv` (source, target, weight) were copied into the database's `import` folder and loaded with Cypher:

```
None
CREATE CONSTRAINT FOR (u:User) REQUIRE u.name IS UNIQUE;


LOAD CSV WITH HEADERS FROM 'file:///centrality_scores.csv' AS row
MERGE (u:User {name: row.user})
SET u.in_degree = toFloat(row.in_degree),
    u.betweenness = toFloat(row.betweenness),
    u.pagerank = toFloat(row.pagerank);


:auto LOAD CSV WITH HEADERS FROM 'file:///edges.csv' AS row
CALL {
  WITH row
  MATCH (a:User {name: row.source}), (b:User {name: row.target})
  MERGE (a)-[:MENTIONS {weight: toInteger(row.weight)}]->(b)
```

```
} IN TRANSACTIONS OF 5000 ROWS;
```

Neo4j reported 72,711 relationships created, matching the NetworkX edge count exactly.

Visualization query (Bloom saved Cypher). Showing all 30,648 nodes is unreadable, so the view keeps the top 10 PageRank users and each one's 30 highest-weight mentioners:

```
None
MATCH (u:User) WITH u ORDER BY u.pagerank DESC LIMIT 10

MATCH (a:User)-[r:MENTIONS]->(u)

WITH u, a, r ORDER BY r.weight DESC

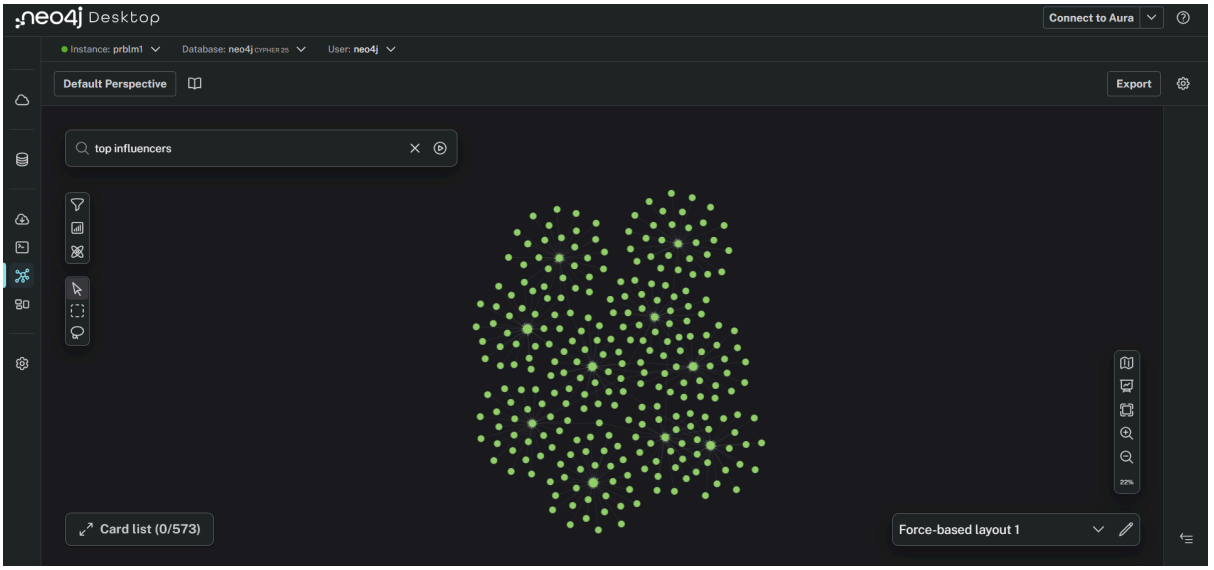
WITH u, collect([a, r])[0..30] AS pairs

UNWIND pairs AS p

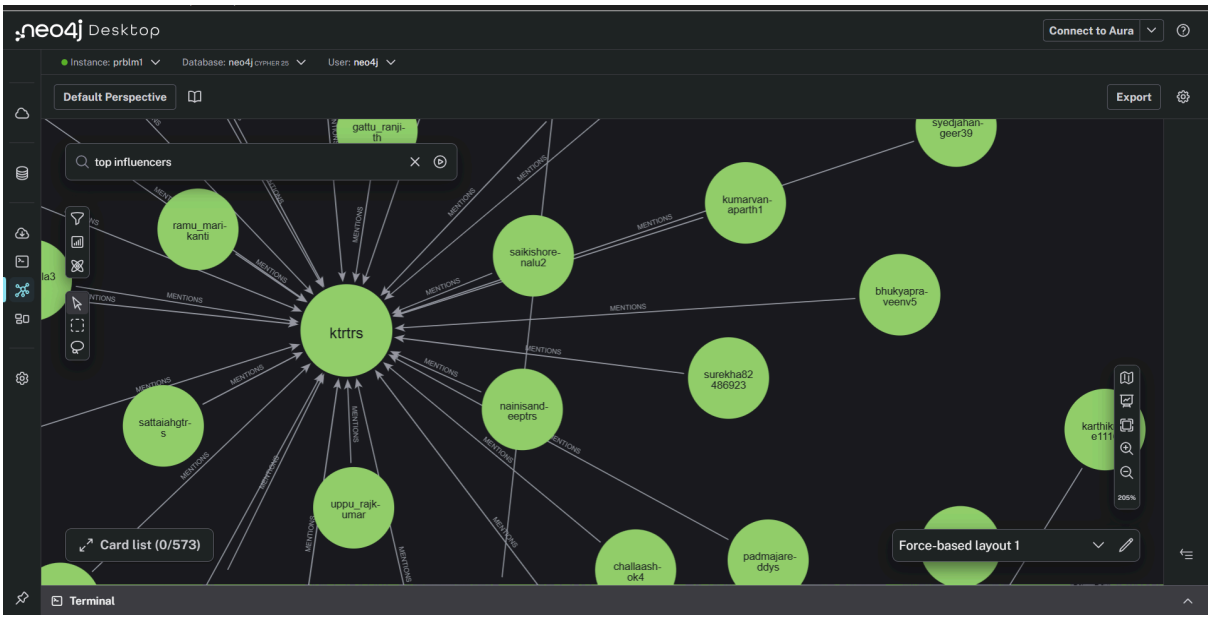
RETURN u, p[0] AS a, p[1] AS r
```

Styling in Bloom: node size scaled by `pagerank` (rule-based, range), and node caption set to `name`.

- Full view of the 10 influencer clusters.



- Zoomed view of one hub (e.g. ktrtrs) with labelled mentioners.



Observations and conclusion

The Telugu mention network is a hub-and-spoke structure dominated by political leaders and parties, with a few film personalities among the KOLs.

1. **Political accounts dominate.** 8 of the top 10 in-degree users are politicians, parties or official handles: Andhra Pradesh (ysjagan, ncbn, pawankalyan, janasenaparty, andhrapradeshcm), Telangana (ktrtrs, trspartyonline) and national leaders (narendramodi, amitshah). kchirutweets is the only film account in the top 10.
2. **In-degree and PageRank largely agree.** The same four users lead both rankings in the same order. Users mentioned by many people are usually also mentioned by well-connected people.
3. **PageRank promotes bjp4telangana.** It is 5th by PageRank (0.0228, almost tied with narendramodi) but not in the in-degree top 10. Its mentioners mention few other accounts or mention it repeatedly, so each mention passes on more weight.
4. **PageRank demotes trspartyonline.** It is 10th by in-degree but drops out of the PageRank top 10, so its mentioners carry less weight.
5. **Betweenness finds bridges, not stars.** The betweenness leaders are ordinary users who both mention others and are mentioned. The big hubs score near zero because they are mostly mentioned and rarely mention back, so few directed paths pass through them.
6. **Betweenness values are tiny (max 0.001).** Most users only post mentions and are never mentioned (average out-degree 2.37, density 7.74×10^{-5}), so the graph has very few long directed paths.
7. **Shared audiences are visible in Neo4j.** Overlapping clusters in Bloom show users who mention several leaders, hinting at party-aligned communities.

Conclusion: the key opinion leaders in this dataset are ysjagan, ncbn and pawankalyan by both popularity and PageRank. PageRank is the most reliable single KOL measure because

it weighs who does the mentioning. Betweenness is better read as a measure of bridge accounts than of influence in a star-shaped mention network.

Limitation: betweenness was approximated from 500 sampled nodes, so its lower-ranked values may shift slightly with a different sample.